

thePowerpuffCards

1 SYSTEM DESIGN

1.1 Initial Design

- The project follows a **client-server architecture**, where the server handles user authentication, card management, battles, and trading.
- A **RESTful API** was designed to handle requests using an HTTP-based communication system.
- PostgreSQL was chosen as the **database management system**, with Docker providing a containerized environment.

2.2 Database Schema

To ensure the efficient storage and retrieval of game data, the following database schema was designed:

Users Table (users)

Stores general player information such as usernames, authentication data, and game currency.

User Profile Table (userprofile)

Contains additional profile information such as bio and profile image.

Cards Table (card)

Stores all collectible cards, classified by type, damage, and element attributes.

Deck Table (deck)

Stores the active deck of each player, ensuring that a deck consists of exactly 4 cards.

Stack Table (stack)

Stores all owned cards of a player, allowing easy retrieval and management.

Packages Table (packages)

Manages the card packages available for purchase and distribution.

Trades Table (trades)

Stores trade offers between players, ensuring fair conditions are met before completing a trade.

Statistics Table (stats)

This table was designed separately from users to improve clarity and performance. It tracks essential game statistics such as games played, games won, and losses, making leaderboard updates and analytics easier to handle.

2 DEVELOPMENT PROCESS

2.1 Implementing Core Features

2.1.1 User Management

- Implemented user registration, authentication, and profile updates.
- Used **JWT-based authentication** for secure API access.

2.1.2 Card System

- Cards are categorized into MonsterCard and SpellCard, both extending the abstract Card class.
- Implemented a database persistence layer for storing and retrieving cards.

2.1.3 Deck Management

- Users can define and configure their **deck** (max 4 cards) using the /deck API.
- Implemented validation to ensure only valid cards are added.

2.1.4 Battle System

- Implemented a **turn-based battle system** where players compete using their deck.
- Used **randomized card selection** and applied boosters dynamically.
- Implemented a **unique doubling feature**, where randomly during battle, a player's selected card might get a damage boost.
- **Doubling Mechanics:**
 - Each player gets **one random round per battle** where a randomly chosen card's **damage is doubled**.
 - The round in which it happens is **randomly determined** at the start of the battle.
 - The doubling effect happens **only once per player per battle**, ensuring unpredictability.
 - The boosted card is **randomly selected** from the player's deck.
- Ensured battle logs provide clear visibility into game progression.

2.1.5 Trading System

- Implemented trading functionality, allowing users to create and accept trade offers.
- Prevented self-trading to avoid exploits.

3 CHALLENGES AND SOLUTIONS

- **Deck duplication after battles**
 - Implemented a proper sequence: `clearDeck()` first, then `saveDeck()` to avoid constraint violations.
- **Ensure players have cards after battles:**
 - Implemented `ensurePlayersHaveCards()` to add random cards if a deck is empty.
- **Authentication failures:**
 - Refined JWT handling to securely validate user sessions.
- **Database connection errors:**
 - Optimized database pooling and added proper error handling.
- **Trade validation issues:**
 - Enforced strict rules for trade eligibility and ownership checks.

4 FINAL IMPLEMENTATION AND UNIT TESTING

4.1 Final Adjustments

- Enhanced logging for better debugging.
- Improved error handling across controllers.
- Conducted load tests to ensure server stability.

4.2 Why These Unit Tests Were Chosen?

- **User Authentication Tests** → Ensures secure login, registration, and token validation.
- **Deck Management Tests** → Verifies the correct addition, removal, and retrieval of cards in a deck to prevent data inconsistencies.
- **Trading System Tests** → Checks the validity of trade offers, preventing self-trading and ensuring fairness.
- **Battle Simulation Tests** → Ensures that battle rounds execute correctly, with damage calculations and card exchanges behaving as expected.

5 LESSONS LEARNED

- **Never underestimate the power of a single missing `clearDeck()` call**
 - I learned the hard way that an extra or missing `clearDeck()` can cause absolute chaos in database integrity.
- **Unit tests are lifesavers**
 - Every bug that took me hours to debug could have been caught early with proper test coverage.
- **Trade validation is like real-world economics**
 - Preventing unfair trades and self-trades turned out to be trickier than expected.
- **Logging is your best friend**
 - Without proper debug logs, I would still be wondering why battles suddenly deleted entire decks.
- **My programming brain functions from 22:00 to 03:00.**
- **Breaks are very important, sometimes even for a few days.**

6 CONCLUSION

The **thePowerpuffCards** project came with its challenges, but through structured debugging, logical fixes, and database optimizations, everything now runs smoothly. The battle system includes a random doubling mechanic, trading is fair, and the ELO ranking keeps things competitive. Future improvements could include AI battle opponents or smarter trade recommendations.